

后端第 3 题：面向对象、封装继承与多态

学号：2026090902009 姓名：吴雨泽

题目：[面向对象、封装继承与多态](#)

本题的五个 Task 都已完成。代码按类分别放在 `src` 下，`OopDemo` 是包含 `main` 的主类；访问权限实验另外使用了 `people` 和 `access` 两个包。这样可以直接比较同包、跨包和子类的访问情况。

运行环境是 macOS、JDK 21，源文件使用 UTF-8 编码。下文的输出取自实际运行，完整结果和编译报错记录放在压缩包的 `验证记录` 目录。

Task 1：类、对象与构造方法

Person 的实现

`Person` 保存姓名、年龄和性别。这里约定 `sex=1` 表示男，`sex=2` 表示女。创建对象时通过构造方法赋值，复制时再创建一个新对象，并用原对象的字段完成初始化。

字段由类内部管理；外部需要修改年龄时调用 `setAge()`，由这个方法检查年龄是否小于 0。吃东西、睡觉、打豆豆是对外提供的行为，设为 `public`；它们共用的打印方法只在 `Person` 内部使用，设为 `private`。代码最后两个观察方法供 Task 2 比较访问范围。

文件：`src/people/Person.java`

```
package people;

public class Person {
    private final String name;
    private int age;
    private final int sex;
    private static int count;

    public Person(String name, int age, int sex) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("姓名不能为空");
        }
        this.name = name;
        setAge(age);
        this.sex = sex;
        count++;
    }

    public Person(Person other) {
        this(other.name, other.age, other.sex);
    }

    public String getName() {
        return name;
    }
}
```

```

    public int getAge() {
        return age;
    }

    public int getSex() {
        return sex;
    }

    public void setAge(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("年龄不能小于 0");
        }
        this.age = age;
    }

    public void printInfo() {
        System.out.printf("姓名=%s, 年龄=%d, sex=%d%n", name, age, sex);
    }

    public void eat() {
        printAction("吃东西");
    }

    public void sleep() {
        printAction("睡觉");
    }

    public void dadoudou() {
        printAction("打豆豆");
    }

    private void printAction(String action) {
        System.out.println(name + "正在" + action);
    }

    // 这两个方法用于比较同包和跨包的访问范围。
    String packageDescription() {
        return name + ", " + age + "岁";
    }

    protected String protectedName() {
        return name;
    }

    public static int getCount() {
        return count;
    }
}

```

创建对象与复制

主类中的主要调用如下：

```
Person original = new Person("小明", 18, 1);
Person copy = new Person(original);
copy.setAge(19);
original.printInfo();
copy.printInfo();
System.out.println(original != copy);
original.eat();
original.sleep();
original.dadoudou();
```

实际运行结果：

```
Task 1: Person 与复制构造
原对象: 姓名=小明, 年龄=18, sex=1
副本: 姓名=小明, 年龄=19, sex=1
两个不同对象: true
小明正在吃东西
小明正在睡觉
小明正在打豆豆
```

副本的年龄变成了 19，原对象仍是 18；`original != copy` 为 true，说明这是两个不同对象。若写成 `Person copy = original`，只是让两个变量引用同一个对象，修改年龄时看到的会是同一份状态。

this 在这里起什么作用

普通构造方法里的 `this.name = name`，左边指当前对象的字段，右边指传入的参数。两者同名时，this 可以明确区分。

复制构造方法里的 `this(other.name, other.age, other.sex)` 则是在调用本类的普通构造方法，复用姓名检查、年龄检查和计数逻辑。本例的 age、sex 是基本类型，name 是不可变的 String，按字段复制就够了。如果以后增加可变的列表字段，还需要决定列表和列表里的对象是否也要独立复制。

类和对象的关系

Person 类规定了一个人有哪些数据、能执行哪些行为；`new Person(...)` 创建的是具体对象。original 和 copy 都属于 Person，但各自保存年龄等实例状态，所以修改其中一个不会自动修改另一个。

Task 2：访问修饰符、实例成员与静态成员

访问权限怎么验证

我用了三种位置进行访问：Person 类内部、同在 people 包里的 SamePackageAccess，以及 access 包里的 OtherPackageAccess。另加 PersonChild 这个跨包子类，用来观察 protected 和普通跨包访问的差别。

Person 的 name、age、sex 都是 private。`printInfo()` 在 Person 内部直接读取这三个字段；外部调用这个公开方法，可以看到它们的值，但不能绕过方法直接修改字段。

同包类：`src/people/SamePackageAccess.java`

```

package people;

public class SamePackageAccess {
    public static void show(Person person) {
        System.out.println("同包访问 public: " + person.getName());
        System.out.println("同包访问 protected: " + person.protectedName());
        System.out.println("同包访问默认权限: " + person.packageDescription());
    }
}

```

跨包普通类: `src/access/OtherPackageAccess.java`

```

package access;

import people.Person;

public class OtherPackageAccess {
    public static void show(Person person) {
        System.out.println("跨包普通类访问 public: " + person.getName());
    }
}

```

跨包子类: `src/access/PersonChild.java`

```

package access;

import people.Person;

public class PersonChild extends Person {
    public PersonChild(String name, int age, int sex) {
        super(name, age, sex);
    }

    public String inheritedName() {
        return this.protectedName();
    }
}

```

合法访问的运行结果:

Task 2: 访问权限与 static
 Person 内部读取 private (通过 printInfo 展示):
 姓名=小明, 年龄=18, sex=1
 通过类名读取创建数量: 2
 同包访问 public: 小明
 同包访问 protected: 小明
 同包访问默认权限: 小明, 18岁
 跨包普通类访问 public: 小明

跨包子类访问 protected: 小林
再创建一个子类对象后, 计数: 3

对于不允许的访问, 我没有把错误代码混进正常项目, 而是分别保存为 `.java.txt` 文件, 再复制成 `.java` 单独编译。下面是编译器实际给出的结论:

访问位置	尝试访问的内容	实际结果
同包的另一个类	<code>person.age</code>	编译失败: age 是 private
同包的另一个类	<code>person.printAction(...)</code>	编译失败: printAction 是 private
跨包普通类	<code>person.packageDescription()</code>	编译失败: 不是公共方法, 无法从外部程序包访问
跨包普通类	<code>person.protectedName()</code>	编译失败: protected 访问限制
跨包子类	通过 Person 类型参数调用 <code>person.protectedName()</code>	编译失败: protected 访问限制

例如, 独立的同包类直接读年龄:

```
package people;

public class PrivateFieldAccess {
    public int read(Person person) {
        return person.age;
    }
}
```

得到的实际报错是:

```
错误: age 在 Person 中是 private 访问控制
    return person.age;
           ^
1 个错误
```

这些失败示例都是独立类。即使 PrivateFieldAccess 和 Person 在同一个包里, private 字段仍不能直接访问。

四种访问权限的结论

修饰符	当前类内部	同包其他类	跨包子类	跨包普通类
public	可以	可以	可以	可以
protected	可以	可以	可以, 但受继承访问规则限制	不可以
默认, 不写修饰符	可以	可以	不可以	不可以
private	可以	不可以	不可以	不可以

protected 不能简单理解成“子类可以通过任何父类对象访问”。本实验中，PersonChild 的 `this.protectedName()` 可以编译；但在跨包子类里，通过一个声明为 Person 的参数调用同一方法，仍会报错。对应失败代码是 `negative-tests/ProtectedReceiver.java.txt`。

Person 的默认方法和 protected 方法专门用于这次对比实验。实际对外提供信息时，使用 public 的 getter；字段保持 private，修改状态也由方法负责检查。

static 与实例成员

`count` 设为 private static，只保留一份，所有 Person 对象共享。构造方法每成功创建一个对象，就把它加 1；复制构造也会调用普通构造，所以复制对象同样会增加计数。

运行时，original 和 copy 创建完以后，`Person.getCount()` 返回 2；再创建一个 PersonChild，父类构造方法也会执行，计数变成 3。这里统计的是累计成功构造次数，不是当前仍然存活的对象数量。

`getCount()` 属于类，且不需要指定某个人的状态，因此可以直接用 `Person.getCount()` 调用。`eat()`、`setAge()` 等实例方法需要知道操作哪个对象，通常先创建对象，再通过对象调用。静态方法没有 this，不能直接用实例字段名来指代某个人的年龄。

Task 3：继承与代码复用

父类与两个子类

父类继续使用题面的 Animal。姓名和编号保存在父类中，子类通过 `super(name, id)` 初始化；介绍和睡觉的方法可以复用，吃东西的方法由两个子类各自重写。

`src/Animal.java`

```
public class Animal {
    private final String name;
    private final int id;

    public Animal(String name, int id) {
        this.name = name;
        this.id = id;
    }

    protected String getName() {
        return name;
    }

    public void eat() {
        System.out.println(name + "正在吃");
    }

    public void sleep() {
        System.out.println(name + "正在睡");
    }

    public void introduction() {
        System.out.println("大家好! 我是" + id + "号" + name + "。");
    }
}
```

```
}  
}
```

src/Penguin.java

```
public class Penguin extends Animal {  
    public Penguin(String name, int id) {  
        super(name, id);  
    }  
  
    @Override  
    public void eat() {  
        System.out.println(getName() + "是企鹅，正在吃鱼");  
    }  
}
```

src/Mouse.java

```
public class Mouse extends Animal {  
    public Mouse(String name, int id) {  
        super(name, id);  
    }  
  
    @Override  
    public void eat() {  
        System.out.println(getName() + "是老鼠，正在吃谷物");  
    }  
}
```

主类用 Animal 引用统一调用：

```
Animal[] animals = {new Penguin("小企鹅", 1), new Mouse("小老鼠", 2)};  
for (Animal animal : animals) {  
    animal.introduction();  
    animal.eat();  
    animal.sleep();  
}
```

实际运行结果：

Task 3: 继承与重写

大家好！我是1号小企鹅。

小企鹅是企鹅，正在吃鱼

小企鹅正在睡

大家好！我是2号小老鼠。

小老鼠是老鼠，正在吃谷物

小老鼠正在睡

两个对象都能调用父类的 `introduction` 和 `sleep`，而 `eat` 的输出不同。循环里的变量虽然声明为 `Animal`，执行 `eat` 时仍会调用实际对象对应的重写方法。

父类的 `private` 字段不能在子类里直接写 `name` 来读取。这里通过 `protected` 的 `getName()` 获取姓名。构造方法本身不会被继承，因此 `Penguin` 和 `Mouse` 都写了自己的构造方法，并在其中调用父类构造方法。

为什么 Java 不支持类的多继承

假设有父类 `A`，`B` 和 `C` 都继承 `A`，并且分别重写了同一个方法；如果 `D` 再同时继承 `B` 和 `C`，调用这个方法时就需要决定采用哪一份实现。两个父类还可能各自带有同名字段和不同的初始化要求，继承关系会更难处理。这就是菱形继承容易引出的歧义。

Java 选择让一个类只有一个直接父类，减少这些冲突。需要具备多种能力时，可以实现多个接口；如果多个接口的默认方法发生冲突，也仍需要在实现类里明确处理。

Task 4：多态与抽象设计

选择接口方案

圆需要半径，矩形需要长和宽，三角形需要三条边；它们的字段不同，但都需要计算面积和周长。因此我用 `Shape` 接口约定这两个方法，每个图形负责自己的计算。

这里的两个接口方法隐含 `public` 和 `abstract`，三个实现类都使用 `@Override`，方便编译器检查方法名和参数是否与接口一致。

src/Shape.java

```
public interface Shape {
    double area();
    double perimeter();
}
```

src/Circle.java

```
public class Circle implements Shape {
    private final double radius;

    public Circle(double radius) {
        if (!Double.isFinite(radius) || radius <= 0) {
            throw new IllegalArgumentException("半径必须为有限正数");
        }
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }

    @Override
    public double perimeter() {
```

```
        return 2 * Math.PI * radius;
    }
}
```

src/Rectangle.java

```
public class Rectangle implements Shape {
    private final double width;
    private final double height;

    public Rectangle(double width, double height) {
        if (!Double.isFinite(width) || !Double.isFinite(height)
            || width <= 0 || height <= 0) {
            throw new IllegalArgumentException("长和宽必须为有限正数");
        }
        this.width = width;
        this.height = height;
    }

    @Override
    public double area() {
        return width * height;
    }

    @Override
    public double perimeter() {
        return 2 * (width + height);
    }
}
```

src/Triangle.java

```
public class Triangle implements Shape {
    private final double a;
    private final double b;
    private final double c;

    public Triangle(double a, double b, double c) {
        if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c)
            || a <= 0 || b <= 0 || c <= 0) {
            throw new IllegalArgumentException("三条边必须为有限正数");
        }
        if (a + b <= c || a + c <= b || b + c <= a) {
            throw new IllegalArgumentException("三条边不能组成三角形");
        }
        this.a = a;
        this.b = b;
        this.c = c;
    }

    @Override
```

```

    public double area() {
        double s = perimeter() / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }

    @Override
    public double perimeter() {
        return a + b + c;
    }
}

```

圆的面积是 πr^2 ，周长是 $2\pi r$ ；矩形的面积是长乘宽，周长是两倍的长宽之和。三角形使用海伦公式：先求半周长 s ，再算 $\text{sqrt}(s * (s-a) * (s-b) * (s-c))$ 。

边长在构造时检查，三角形还要满足任意两边之和大于第三边，避免先创建一个不成立的图形，再在计算面积时得到错误结果。

用同一组方法调用不同图形

```

List<Shape> shapes = List.of(
    new Circle(2), new Rectangle(3, 4), new Triangle(3, 4, 5));
for (Shape shape : shapes) {
    System.out.printf(Locale.R00T, "%s: 面积=%.3f, 周长=%.3f\n",
        shape.getClass().getSimpleName(), shape.area(), shape.perimeter());
}

```

实际运行结果：

```

Task 4: 图形多态
Circle: 面积=12.566, 周长=12.566
Rectangle: 面积=12.000, 周长=14.000
Triangle: 面积=6.000, 周长=12.000
创建边长 1、2、3 的三角形：三条边不能组成三角形

```

三种图形放在 `List<Shape>` 中，循环里始终调用 `area` 和 `perimeter`，不需要分别写“如果是圆就用圆的公式”。`getClass().getSimpleName()` 只用于显示类名，具体计算由对象的实现方法完成。这就是这段代码中体现的多态。

本题采用接口方案，符合“至少一种”的要求。如果以后需要所有图形共享颜色、位置等字段和初始化过程，再考虑使用抽象类；目前共同部分主要是计算能力，接口更直接。

Task 5：封装与接口设计

可见性与对外接口

账户的账号、户名、余额和密码都设为 `private`。账号、户名和密码由构造方法确定；余额会变化，但只能通过存款、取款、转账方法修改，没有直接设置余额的 `setter`。

构造方法和题面列出的业务方法设为 public，调用者可以使用这些功能。内部校验方法设为 private，调用者不需要直接使用。getAccountInfo() 返回遮掩后的账号、户名和余额，不返回密码。

题面将两个校验方法标为“只需修改可见性”。这里保留它们的 return true 占位实现，只改成 private；实际的密码和金额判断写在业务方法里，不能把这个占位返回值当成已经通过校验。

BankAccount 完整实现

src/BankAccount.java

```
import java.util.Locale;

public class BankAccount {
    private final String accountNumber;
    private final String accountHolder;
    private double balance;
    private final String password;

    public BankAccount(String accountNumber, String accountHolder,
                        double initialBalance, String password) {
        if (accountNumber == null || accountNumber.isBlank()
            || accountHolder == null || accountHolder.isBlank()
            || password == null || password.isEmpty()) {
            throw new IllegalArgumentException("账号、姓名和密码不能为空");
        }
        if (!Double.isFinite(initialBalance) || initialBalance < 0) {
            throw new IllegalArgumentException("初始余额必须为有限非负数");
        }
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
        this.password = password;
    }

    public void deposit(double amount) {
        if (!Double.isFinite(amount) || amount <= 0
            || !Double.isFinite(balance + amount)) {
            throw new IllegalArgumentException("存款金额必须为有限正数，余额不能溢出");
        }
        balance += amount;
    }

    public boolean withdraw(double amount, String inputPassword) {
        if (!password.equals(inputPassword)
            || !Double.isFinite(amount) || amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    public boolean transfer(BankAccount recipient, double amount, String inputPassword) {
```

```

        if (recipient == null || recipient == this
            || !password.equals(inputPassword)
            || !Double.isFinite(amount) || amount <= 0 || amount > balance
            || !Double.isFinite(recipient.balance + amount)) {
            return false;
        }
        balance -= amount;
        recipient.balance += amount;
        return true;
    }

    public double getBalance() {
        return balance;
    }

    public String getAccountInfo() {
        String hiddenNumber = accountNumber.length() > 4
            ? "****" + accountNumber.substring(accountNumber.length() - 4) : "****";
        String hiddenHolder = accountHolder.length() > 1
            ? accountHolder.substring(0, 1) + "*" : "*";
        return String.format(Locale.ROOT, "账号=%s, 户名=%s, 余额=%.2f元",
            hiddenNumber, hiddenHolder, balance);
    }

    // 按题面“只需修改可见性”保留占位实现。
    // 实际密码和金额检查已写在上面的业务方法中。
    private boolean validatePassword(String inputPassword) {
        return true;
    }

    private boolean validateAmount(double amount) {
        return true;
    }
}

```

存款方法是 void，遇到非法金额就抛出异常，明确拒绝本次操作。取款和转账返回 boolean，密码错误、金额不合适或余额不足时返回 false，余额保持原样。

转账还会检查收款人是否为空、是否是当前账户，以及收款后余额是否会溢出。所有检查通过后，才扣除付款账户余额并增加收款账户余额。金额除正数外，还要求是有限数，避免 NaN 或 Infinity 混入余额计算。

操作后的余额是否正确

实验使用张三和李四两个测试账户，初始余额分别为 100 元和 0 元，密码也是练习数据。流程是：存入 20 元、取出 50 元、转出 30 元，然后尝试错误密码、余额不足和非法金额等情况。

实际运行结果：

Task 5: 银行账户封装

初始余额：付款账户=100.00元，收款账户=0.00元

存入20元：付款账户=120.00元，收款账户=0.00元

取出50元：true

```
取款后：付款账户=70.00元，收款账户=0.00元
转出30元：true
转账后：付款账户=40.00元，收款账户=30.00元
错误密码取款：false，两账户余额=40.00 / 30.00元
余额不足取款：false，两账户余额=40.00 / 30.00元
错误密码转账：false，两账户余额=40.00 / 30.00元
余额不足转账：false，两账户余额=40.00 / 30.00元
负数取款：false，两账户余额=40.00 / 30.00元
零金额转账：false，两账户余额=40.00 / 30.00元
NaN金额转账：false，两账户余额=40.00 / 30.00元
空收款人：false，两账户余额=40.00 / 30.00元
向自己转账：false，两账户余额=40.00 / 30.00元
零金额存款：已拒绝，余额=40.0
账户信息：账号=****5678，户名=张*，余额=40.00元
全部运行检查通过
```

成功操作后，付款账户余额为 $100 + 20 - 50 - 30 = 40$ 元，收款账户余额为 30 元。各次失败操作后，两边仍为 40 元和 30 元；主类还对这两个值进行检查，如果失败请求改变了余额，程序会抛出 `AssertionError`。

最后返回的账户信息只保留账号末四位和姓名首字，没有输出密码。

外部能否直接修改余额

用独立类尝试修改 `private` 字段：

```
public class BalanceAccess {
    public void change(BankAccount account) {
        account.balance = 10000;
    }
}
```

实际编译结果：

```
错误：balance 在 BankAccount 中是 private 访问控制
    account.balance = 10000;
        ^
1 个错误
```

这说明外部不能绕过账户接口直接改余额。`private` 负责限制直接访问，业务方法负责判断操作是否允许；两者一起才能保护账户状态。

类之间的关系与设计取舍

- `OopDemo` 创建并调用 `Person`、`Animal`、`Shape` 和 `BankAccount`，负责展示各个实验。
- `Penguin`、`Mouse` 继承 `Animal`，复用共有状态和行为。
- `Circle`、`Rectangle`、`Triangle` 实现 `Shape`，向调用者提供统一的计算方法。
- `PersonChild` 用于跨包 `protected` 实验，和普通跨包类作对照。

这次把每个主要类拆成独立文件，访问权限的边界比较清楚，也能避免嵌套类影响 `private` 实验。`Person` 只允许修改年龄，姓名和性别在构造时确定；`Shape` 的各个尺寸也在构造后保持不变，计算时不需要再担心外部改出非法边长。

账户余额沿用题面的 `double`，仍会有小数精度限制。如果继续完善这部分，我会先改用 `BigDecimal`，并明确金额的小数位和舍入规则。这份代码的范围是题目中的内存操作实验。

如何运行与核对

在压缩包里的 `03-oop` 目录打开终端：

```
bash run.sh
```

它执行的编译和运行命令是：

```
mkdir -p build
javac -encoding UTF-8 -d build src/*.java src/people/*.java src/access/*.java
java -cp build OopDemo
```

验证不允许的访问：

```
bash verify-access.sh
```

这个脚本会逐个编译 `negative-tests` 中的六份示例，预期每份都失败。它们保存为 `.java.txt`，不会影响正常程序的编译。完整的实际结果见包里的 `验证记录/访问权限编译结果.txt`；正常程序的完整输出见 `验证记录/程序运行结果.txt`。